

第28讲：从 LLM 到 Agent

Planning、State、Memory、Tools、Environment 与 Action Loop

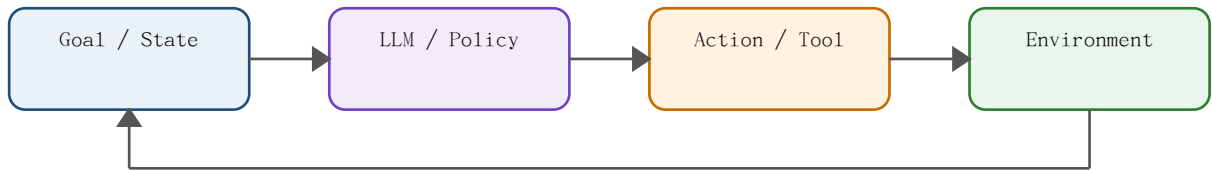
本讲目标

把前面 27 讲第一次完整组装起来。学完后你应该能回答：LLM 和 Agent 到底差在哪里；一个 Agent 为什么需要 Goal、State、Planning、Memory、Tools、Environment 与循环；Agent 的“自主性”具体来自什么；以及一个看似聪明的 Agent 为什么仍然可能因状态、计划、工具或停止条件失败。

普通 LLM：一次输入 → 一次输出



Agent：持续 Observe → Decide → Act → Observe



先记住一句：LLM 是一个模型；Agent 是一个围绕模型构造的持续决策系统。

1. LLM 和 Agent 到底差在哪里？

普通 LLM 最典型的交互是：给一个 Context，做一次或一段连续 token 生成，然后结束。Agent 则把 LLM 放进一个循环里，让它不断读取状态、做决定、调用工具、观察结果，并根据新状态继续行动。

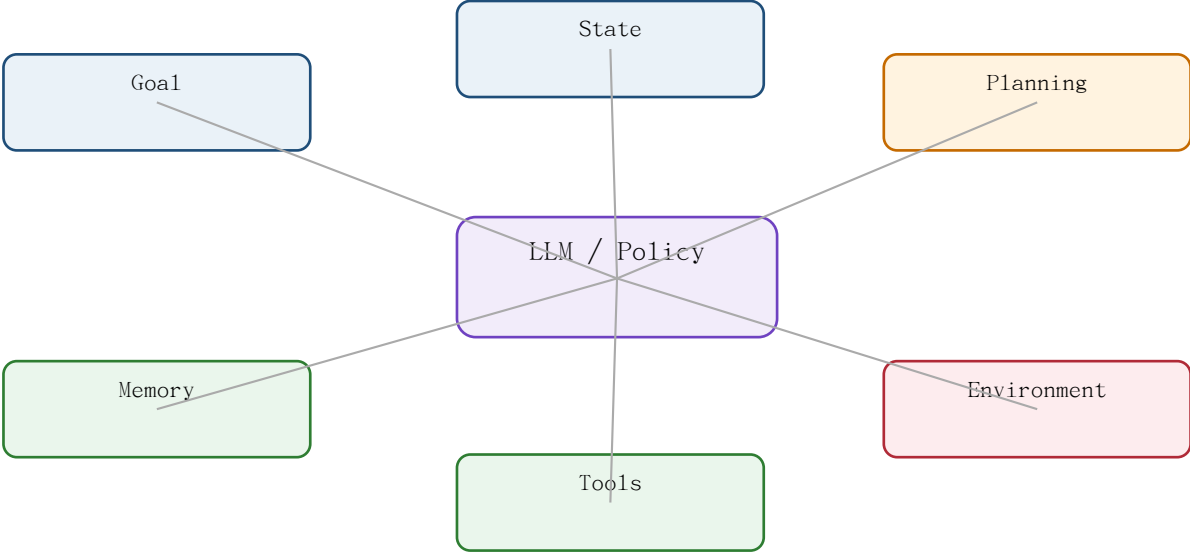
维度	普通 LLM	Agent
核心对象	概率语言模型	决策与执行系统
输入	Prompt / Context	Goal + State + Observation
输出	文本 / token	Action / Tool Call / Message / Final Answer
是否持续循环	通常不是	通常是
是否读取环境反馈	有限	核心能力之一
是否维护状态	主要依赖 Context	显式 State + Context + Memory
是否会真实行动	本身不会	可通过 Runtime / Tools 行动

关键

Agent 不是一种新的神经网络架构。很多 Agent 仍然使用普通 Decoder-only LLM，只是外面多了一层 Runtime、State、Memory、Tool 和控制逻辑。

2. 一个最小 Agent 至少包含什么？

一个 Agent 不是 “一个更大的 Prompt”，而是一组持续交互的系统组件。



组件	作用
Goal	最终要达到什么状态
State	系统当前知道 “现在是什么情况”
Policy / LLM	根据 Goal + State 决定下一步
Planning	把长期目标分解成步骤/子目标

组件	作用
Memory	保存过去经历或长期知识
Tools	能够执行外部动作
Environment	动作真正发生并产生反馈的世界
Stop Condition	什么时候任务算完成/失败/超预算

3. Goal: Agent 为什么必须知道“最终要干什么”？

没有 Goal，系统就无法判断某个 Action 是不是有进展，也无法判断什么时候停止。

例如：

```
Goal: 读取 data.csv 并生成 summary.txt
```

这个 Goal 可以继续拆成：

```
locate file -> read data -> compute mean -> write output -> verify file
```

误区

“给 Agent 一段长 Prompt” 不等于有明确 Goal。真正可执行的 Goal 应该能映射到可观察的完成条件。

4. State: Agent 为什么必须维护“现在到哪一步了”？

State 是 Agent 当前决策真正依赖的工作状态。它可能包括：

State 类型	例子
Task State	当前完成了哪些子任务
Environment State	文件是否存在、网页状态、API 返回值
Tool State	哪些工具可用、是否限流
Resource State	剩余 token / 时间 / 调用预算
Conversation State	用户补充了什么约束
Memory-derived State	过去经历告诉我们什么

如果 State 错了，后面的 Planning 再强也会建立在错误世界模型上。

例子

Agent 误以为“文件已经下载完成”，于是跳过下载直接解析；后续就可能出现 FileNotFoundError。这个失败首先是 State Tracking 问题，不一定是语言推理能力问题。

5. Planning: 把目标变成可执行步骤

Planning 的本质是从当前 State 出发，构造一条可能到达 Goal 的 Action 序列。

最简单可以表示成：

```
Plan = [Action_1, Action_2, ..., Action_n]
```

但真实环境会变化，所以 Plan 往往不是一次生成后永远不改。

规划方式	特点
Plan-then-Act	先给完整计划，再逐步执行
ReAct	Reason/Act/Observe 交替，边走边修正
Replanning	工具失败或状态变化后重新规划
Search / Tree	尝试多个候选分支，再选择

核心

Planning 不是“输出更多思考文字”。真正的规划必须和 State、Action、Environment Feedback 连起来。

6. ReAct：为什么它很适合解释 Agent Loop？

ReAct 可以用一个非常直观的循环表示：

```
Reason -> Act -> Observe -> Reason -> Act -> Observe -> ...
```

例如：

```
Reason: 看看 data.csv 文件 Act: file_exists("/workspace/data.csv") Observe: true Reason:
读取 csv 文件 Act: read_csv(...) Observe: rows=100, ... ...
```

关键不是“Reason”这个词，而是每次行动后都有新的 Observation，下一步决策可以利用真实反馈。

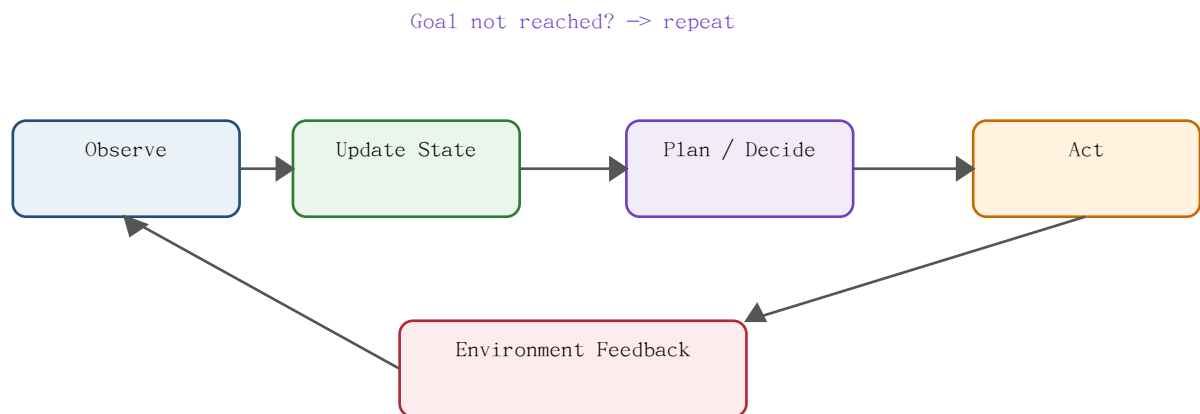
7. Environment：Agent 的“世界”是什么？

Environment 是 Agent 行动真正作用的外部系统。它不一定是机器人世界，也可以是：

Environment	可能的 Action
文件系统	读取、创建、修改文件
浏览器/Web	搜索、点击、填写表单
代码环境	执行程序、跑测试
数据库	查询、更新记录
企业系统	工单、CRM、日历、邮件
模拟环境	游戏、GridWorld、benchmark

Environment 提供的最重要东西是 Feedback：动作到底成功没有、世界现在变成了什么。

8. Action Loop: Agent 真正“活起来”的地方



Stop 条件同样是 Agent 设计的一部分。

一个最小 Agent Loop 可以写成:

```
while not done: observation = environment.observe() state = update_state(state, observation)
action = policy(goal, state, memory, tools) result = environment.execute(action) done =
check_stop(goal, state, result)
```

注意: 其中只有 `policy(...)` 可能由 LLM 完成。其他部分大量属于普通软件系统。

这就是 AI Systems 视角
Agent 的可靠性来自“模型能力 + Runtime 控制 + State 管理 + 工具反馈 + 停止条件”的整体，而不是只看 LLM benchmark 分数。

9. Memory 在 Agent Loop 中到底放在哪里?

Memory 通常不直接代替 State，而是为当前 State 提供过去经历。

Memory Store -> Retrieve relevant past experience -> Inject into Context -> Policy decides	
State	Memory
描述“现在”	保存“过去”
通常需要高度准确	可以很大、需要检索
直接影响当前决策	通过 Retrieval 间接影响
任务结束后部分状态失效	可跨任务长期保存

因此第25讲讲过的 Write / Retrieve / Update / Forget Policy 会直接进入 Agent 的长期行为。

10. Tools 在 Agent 中是“动作空间”

强化学习里常用 Action Space 表示 Agent 可以做哪些动作。LLM Agent 的 Tool Set 可以看作一种离散/结构化 Action Space。

例如一个研究 Agent 的 Action Space 可能是:

search_paper / open_pdf / run_python / write_note / stop

工具越多并不自动越强：Action Space 变大后，Tool Selection 也会更难。

连接第27讲

Tool Schema 定义了动作长什么样，Runtime 负责校验并执行。Agent 只是在循环中反复选择这些动作。

11. Stop Condition: 为什么 Agent 也会“停不下来”？

如果只写 `while True`，Agent 可能陷入重复搜索、反复修改、工具循环，直到耗尽 token 或预算。

Stop 条件	例子
Goal achieved	summary.txt 已存在且内容验证通过
Max steps	最多 10 步
Budget exhausted	API 成本达到上限
Timeout	总运行时间超过限制
No progress	连续 N 步状态没有改善
Fatal error	权限不足且无法恢复
Human handoff	需要用户决策

系统原则

“什么时候停止”与“下一步做什么”同样重要。无限重试不是自主性，而是控制失效。

12. Agent 的自主性到底是什么？

自主性不是神秘属性，可以拆成几个可观察维度：

维度	低自主性	高自主性
规划范围	用户指定每一步	自己分解长期目标
工具选择	固定脚本	动态选择工具
状态更新	人工告诉	自动读取环境
错误恢复	失败就停止	根据反馈重新规划
持续时间	单步	多轮持续执行
权限	只读/沙盒	可影响真实系统

自主性越高，潜在能力更强，但错误传播和治理难度也更高。

13. Agent 失败不一定是 LLM “不会推理”

失败层	典型现象
Goal specification	目标含糊或不可验证

失败层	典型现象
State tracking	误判当前状态
Planning	步骤顺序错误/遗漏必要子任务
Tool selection	选错工具
Arguments	工具参数错误
Execution	工具超时/权限失败
Memory	检索到过期/错误经验
Observation parsing	误读 Tool Result
Stop condition	过早结束或无限循环

科研方法

Agent benchmark 最好记录 trajectory，而不只记录最后成功/失败。没有轨迹就无法定位是哪一层导致错误。

14. Agent Trace / Trajectory: 为什么轨迹是科研数据？

一次 Agent 运行不是只有 Final Answer，而是一条状态转移轨迹：

```
s0 -> a0 -> o1 -> s1 -> a1 -> o2 -> ... -> terminal
```

其中：

符号	含义
s _t	第 t 步状态
a _t	第 t 步动作
o _t	环境反馈/Observation
terminal	成功、失败或停止

Trajectory 可以用来研究：错误从哪一步开始、是否能够恢复、哪些 Action 最消耗预算、不同 Agent 是否形成不同策略。

15. Agent Evaluation: 最终成功率远远不够

指标	测什么
Task Success Rate	最终任务是否完成
Step Count	用了多少步
Tool Calls	调用了多少次外部工具
Token / Cost	花了多少模型预算
Latency	完成任务花了多久
Recovery Rate	中途失败后能否恢复
Invalid Action Rate	非法动作比例
State Accuracy	内部状态是否与环境一致
Plan Adherence / Replanning	是否合理执行/调整计划

Compute-matched
比较两个 Agent 方法时，最好同时报告相近的 Tool Call / Token / Time Budget。否则“更高成功率”可能只是花了更多 test-time compute。

16. 单 Agent 的几个典型架构

架构	特点	适合
Direct Agent	每步直接决定下一 Action	简单工具任务
Plan-then-Act	先生成计划再执行	流程较稳定任务
ReAct	边推理边行动边观察	动态环境
Planner-Executor	Planner 规划，Executor 执行	复杂长任务

架构	特点	适合
Agent + Verifier	执行后有独立检查	高可靠任务

这些名字在不同论文和框架里可能不完全一致，关键还是看真实的数据流和控制流。

17. Framework 不是 Agent 的科学本体

LangChain、AutoGen、CrewAI 等框架可以帮助你快速搭建 Agent，但科研时不要把框架 API 当成研究对象本身。

真正的研究变量是：

State / Planning / Memory / Tool Policy / Communication / Feedback / Verification / Budget
科研视角 框架只是实现手段。论文要描述机制，而不是只说“我们用了某框架的 Agent”。

18. 从单 Agent 到多 Agent：多出来的是什么？

多个 Agent 不是简单复制多个 LLM 请求。系统还增加：

新增组件	问题
Role / Specialization	每个 Agent 负责什么
Communication	谁和谁说什么
Shared State	哪些信息全局共享
Private State / Memory	哪些经历只属于自己
Coordination	如何分工、同步、冲突解决
Aggregation	最后怎样形成集体决策
Resource Allocation	预算怎样分给不同 Agent

下一阶段 Agent Systems 课程会深入这些问题；本讲只需要知道：多 Agent 是在单 Agent Loop 外再加一层交互系统。

19. 与你的研究方向连接：Agent 是一个动态系统

从 AI Systems Science 视角，一个 Agent 可以抽象成：

State _t + Observation _t + Memory _t -> Policy -> Action _t -> Environment -> State _(t+1)

因此研究问题自然包括：

问题	例子
可预测性	同样初始状态下轨迹有多稳定？
错误传播	一个错误会影响多少后续步骤？
恢复能力	错误后能否回到正确轨迹？
资源效率	成功需要多少 token/tool calls？

问题	例子
长期变化	Memory 如何让策略随经历变化?
交互效应	多个 Agent 的错误如何相关/放大/抵消?
这就是核心转变 研究对象不再只是“模型能不能答对一道题”，而是“有限、随机、会犯错的 AI 组件组成的系统怎样随时间演化”。	

20. 小实验：一个最小 Agent Loop

材料包里的 Python 实验构造了一个极小工作环境。目标是：

Goal: read data.txt -> compute mean -> write summary.txt -> verify output

Environment 提供四个工具：

file_exists / read_numbers / write_summary / verify_summary

Toy Policy 会根据 State 决定下一 Action。实验同时故意制造一次可恢复错误：第一次写 summary 时写错值，Verifier 发现错误，Agent 更新 State 后重新计算并修正。

你要观察什么

重点不是这个手写 Policy 有多聪明，而是看完整闭环：Observation 改变 State，State 改变 Action，Tool Result 再改变下一轮决策。

21. 常见误区汇总

误区	更准确的理解
Agent = 更长 Prompt	Agent 的关键是状态、动作和反馈循环
Agent = Tool Calling	Tool Use 只是 Agent 的一个组件
有 Planning 就一定更强	计划可能错，也可能不适应动态环境
Memory 越多 Agent 越聪明	错误/过期 Memory 会干扰决策
能自主循环就等于智能	也可能只是自动重复错误
最终成功就说明过程合理	可能浪费大量 token/tool calls
多 Agent 一定优于单 Agent	必须控制预算与错误相关性

22. 练习题

- 1) LLM 和 Agent 最根本的区别是什么？
- 2) 为什么 Agent 必须显式或隐式维护 State？
- 3) Planning 和普通“多写一些推理文字”有什么区别？
- 4) ReAct 的核心闭环是什么？
- 5) 为什么 Environment Feedback 对 Agent 很关键？
- 6) Memory 和 State 有什么区别？
- 7) 为什么 Stop Condition 是 Agent 设计的一部分？
- 8) 为什么 Agent benchmark 应该保存 trajectory？
- 9) 怎样区分“Agent 更聪明”和“只是用了更多 tool calls / tokens”？
- 10) 把一个单 Agent 扩展成多 Agent，系统层面会新增哪些问题？

答案见材料包《03_第28讲_练习题与答案.pdf》。

23. 本讲总结

一句话总纲

Agent = LLM/Policy + Goal + State + Planning + Memory + Tools + Environment + Feedback Loop + Stop Condition。它不是单个模型，而是一个随环境反馈不断进行状态转移的动态系统。

组件	最短记法
Goal	要到哪里
State	现在在哪里
Planning	准备怎么走
Policy / LLM	下一步选什么
Tools / Actions	能做哪些事
Environment	动作真正发生的世界
Feedback	动作之后实际发生了什么
Memory	过去有哪些可利用经历
Stop	什么时候结束

下一讲：第29讲《单 Agent 与多 Agent：多个 Agent 到底多了什么？——角色、通信、共享状态、协调与集体决策》。